Learning What to Change: the science-first poster, explained

Speaker explainer for poster_science_first_review.pdf

Chimdi Walter Ndubuisi, University of Missouri

19 October 2026 · REVIEW DRAFT for collaborator review, not for circulation

How to use this. Read the plain text aloud when presenting. Every figure is reproduced from recorded, hash-pinned data, or it says ILLUSTRATIVE on its face. Every number comes from the claim ledger (`records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`), with its scope. The poster is a review draft for Toni, Chris and Pierce; nothing here is a print-approved claim.

1 The question, and why it is hard

The general question. How can a machine learn the objects, relationships and rules in unfamiliar data, and know what to reconsider when its explanation is not enough? The hard part is not only solving a problem. Before any rule can be found, the system must decide what the entities are and how they relate. After a rule is found, it must tell whether the rule really explains the observations, and when it does not, which part of its explanation to change.

This is a research question, not an achievement. The system described here, CORA, makes parts of it executable and measurable in one controlled setting. It has not solved the general problem, and it has not been applied to biology.

The 20-second version. I study how a reasoning system can tell what to change when its explanation is not good enough: the objects it sees, the rule it uses, or simply the evidence it has. CORA solves ARC puzzles by writing small programs. It once built a new rule on a real puzzle and checked it carefully, but a version that ignored its failure record did the same, and on 58 more puzzles construction added nothing. The measured bottleneck is proposing a usable rule at all. Whether a system can learn which part of an explanation to revise is the open question.

The 90-second version. Start at the top of the poster with the question and the leaf. Then follow the loop from left to right: observations, objects, executable rules, testing, and the decision whether the account is supported. Point to the two shaded bottlenecks. Then show the ARC grids: what a task is, the rule CORA built, and the 0 of 58. End with the dashed box: a learned prior inside CORA's own search, proposed and not yet validated. Full scripts: `POSTER_90S_SCIENCE_FIRST.md` and `POSTER_3MIN_SCIENCE_FIRST.md`.

2 A biological motivation

The poster's leaf is an **illustrative schematic**, not a photograph and not data. It stands for a common situation in biology. Maize lesion-mimic plants form spots of dead tissue without a pathogen. An image shows where and how the lesions form: their size, spacing and spread. It does not show which biological process produces them.

- **Pattern is not mechanism.** Image geometry and topology constrain the possibilities. Naming a process would need independent molecular or perturbation evidence.
- **The entities are part of the inference.** Should the unit be a lesion, a cluster of lesions, a leaf region, a cell state or a plant? The answer changes which rule can be stated at all.

- **The level of inference matters.** Conclusions drawn at the level of single measurements, of individuals, or of experimental conditions can disagree. A trustworthy system must say at which level a claim holds.
- **Independent checks decide.** An explanation that fits the images is a hypothesis. It becomes evidence only when it predicts something checked independently.

No biological pathway, mechanism or molecular result is claimed here. Any biological test would be designed with the collaborators who hold the data and the domain knowledge (section 11).

3 The reasoning loop and its two bottlenecks

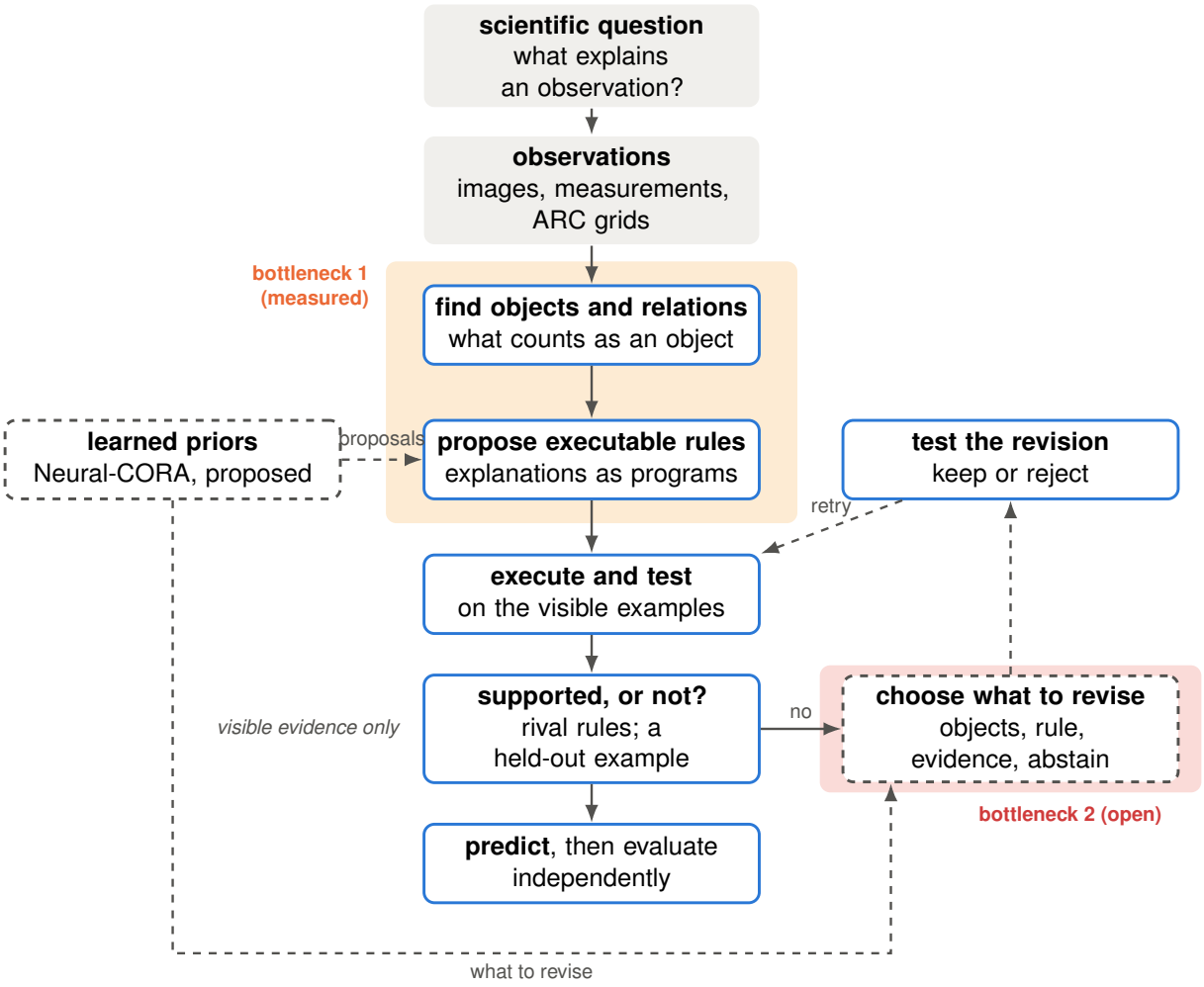


Figure 1: The poster's reasoning loop. Solid: implemented and tested in CORA. Dashed: proposed or not yet validated. Grey: context, not a CORA component. Shading marks the two bottlenecks.

The loop in figure 1 is the poster's main figure. Each arrow means:

1. **Scientific question → observations.** A question picks the data: images, molecular measurements, or, in this work, ARC input and output grids. Grey boxes are context, not parts of CORA.
2. **Find objects and relations.** Decide what counts as an object and which relations matter. In CORA this is one of seven segmentation views of a grid. This is the **representation**.
3. **Propose executable rules.** Write an explanation as a program that can be run: a **rule**. CORA proposes programs by searching its typed language, by native induction from object changes, or by constructing a missing operation.

4. **Execute and test.** Run each rule on every demonstration. A rule that reproduces every demonstration exactly **fits**.
5. **Supported, or not?** A fitting rule may still be unsupported. Rival rules may fit and disagree on the new input, or a rule may fail when one demonstration is held out and rebuilt from the rest (**insufficient evidence**). This decision uses visible evidence only: CORA never sees the hidden test answer.
6. **Predict, then evaluate independently.** A supported rule predicts the new output. A separate evaluator, which alone reads the answers, scores it later.
7. **Choose what to revise.** If the account is incomplete, conflicting or fragile, decide whether to change the objects, the rule, or the evidence and search, or to abstain. This choice is **not** validated (dashed).
8. **Test the revision.** Install the change reversibly, re-test it, and keep it only if it passes the same checks. A tested revision is not automatically a correct explanation.
9. **Learned priors (proposed).** A trained model would rank proposals and revisions **inside** this loop. It never produces an answer by itself (section 9).

Bottleneck 1 is measured. Most failures happen before any test: CORA often cannot propose a usable rule. On 58 separately selected tasks, construction proposed no rule for 55 (section 7). **Bottleneck 2 is open.** Even when a failure is visible, nothing yet shows that a system can learn which part to revise better than strong blind alternatives.

An observable diagnostic is not knowledge of the hidden answer. CORA can see that two of its rules disagree, or that one demonstration alone supports a rule. It cannot see whether its prediction is right.

4 Why ARC-AGI-2 is the testbed

ARC-AGI-2 is a deliberately simplified environment. Each task gives two to five demonstration pairs (an input grid and its output grid) and one new input. The solver must produce the new output exactly, with two attempts. The answer is exact, the evidence is small, and every rule can be executed and checked, so representation, rule and evidence can be studied one at a time.

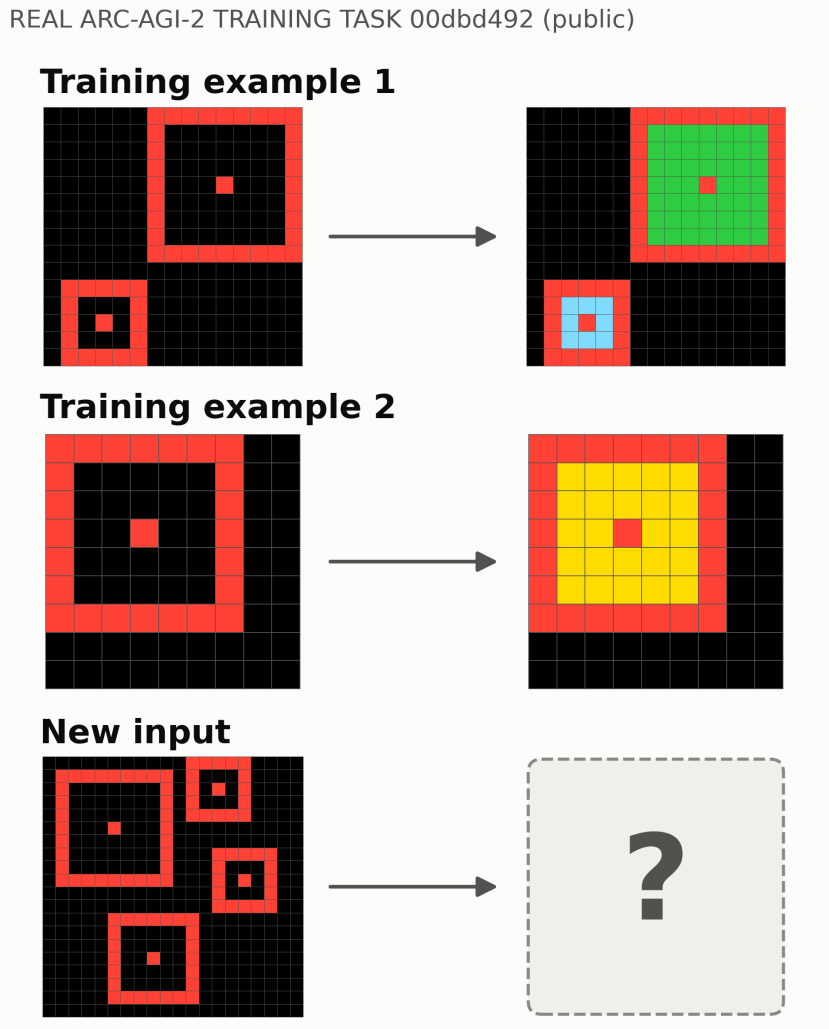


Figure 2: A real public training task, 00dbd492, drawn cell for cell from the recorded bundle. Each frame is filled with a colour set by its size. The task was examined during development; here it only shows what an ARC task is.

Memorization versus generalization. "Paint the cells that changed in example 1" fits example 1 and fails elsewhere: it memorizes. "Fill each frame with the colour its size calls for" generalizes. Both fit the demonstrations they were built from, and that is the difficulty: fitting is not explaining.

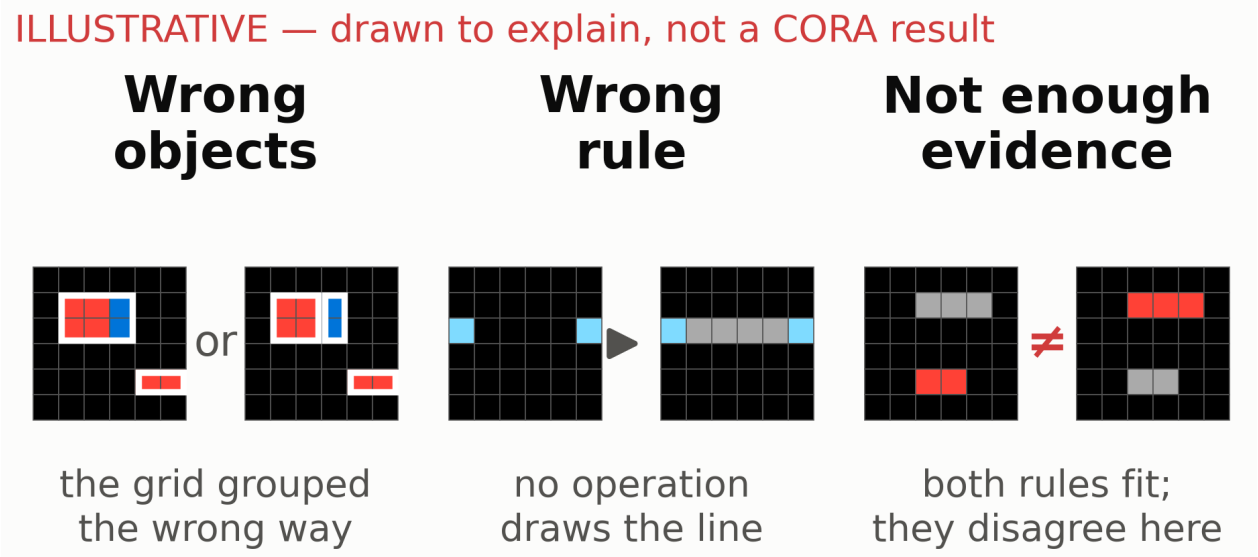


Figure 3: ILLUSTRATIVE: three possible causes of a failed explanation, drawn to explain, not ARC tasks and not CORA results. Wrong objects: the same grid splits into two or three objects. Wrong rule: the available operations (move, recolour) cannot draw the line. Not enough evidence: "bars of two turn red" and "the top bar turns red" fit the same example and disagree on a new input.

5 What CORA implements today

CORA is an executable reasoner. Every hypothesis is a program that a fixed interpreter runs on the demonstrations.

- **Native CORA (frozen v23).** It segments a grid into objects in seven views, searches for an object program (select objects, then keep, delete, recolour, move, copy, grow, . . .), and accepts only programs that reproduce every demonstration and are rebuilt when each demonstration is held out in turn.
- **Rule construction (CORA-FHR, failure-conditioned hypothesis revision).** It reads native's own failure trace and constructs a missing growth operation from a fixed typed grammar of 37,750 expressions. It installs the operation reversibly and keeps it only if the unchanged engine uses it and every held-out fold rebuilds it.
- **CORA-v2, the hypothesis reasoner.** It keeps every hypothesis (representation, geometry, colour policy) that reproduces every demonstration. It abstains when they disagree, reruns the search with each demonstration held out, and labels **why** it is unsure with observable diagnoses. Exploratory guesses are kept apart from certified answers.
- **Present versus proposed.** Solid in figure 1: objects, executable rules, execution, the support check, the tested revision and the separate evaluator. Dashed: choosing which revision to make, and any learned prior.

6 Recorded evidence: a rule CORA built

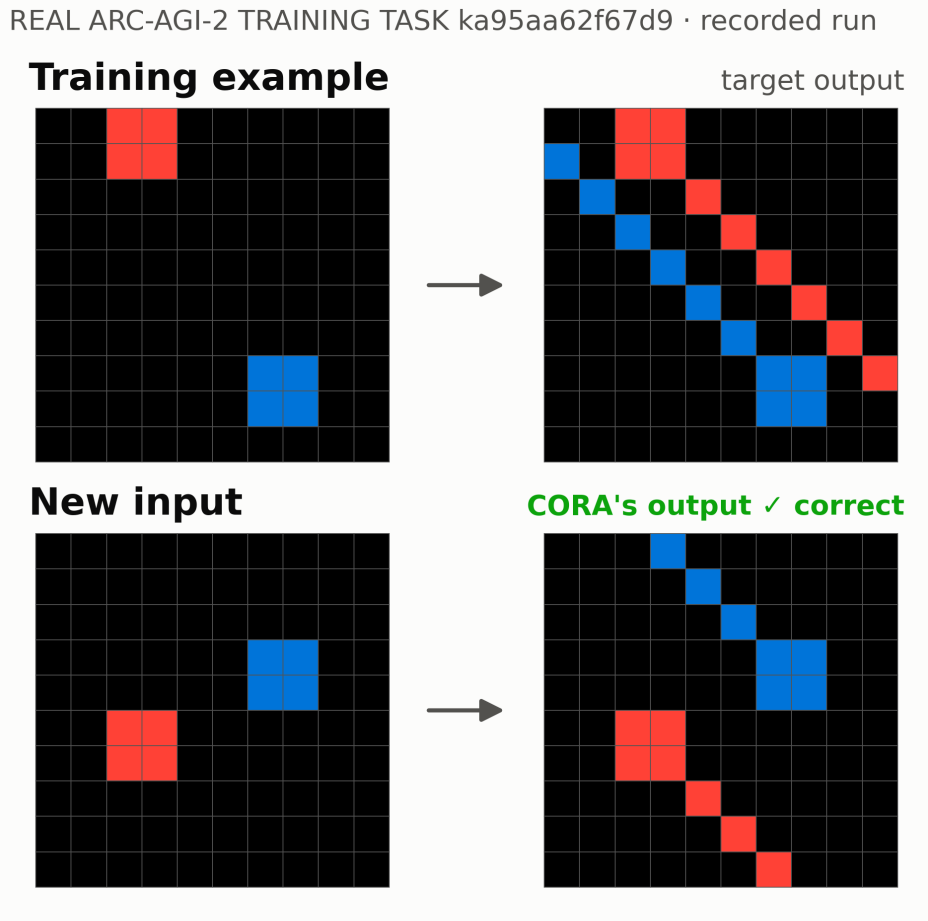


Figure 4: Real grids from the recorded bundle of task ka95aa62f67d9 (a training example with its target output; the new input with CORA's output, which equals the evaluator's answer).

- **The rule CORA constructed.** If an object's colours are {1}, a ray goes north-west from its centre to the border; otherwise south-east. In ARC colours: **blue objects send a line up-left, red objects down-right**, each in the object's own colour.
- **The checks.** The rule was installed reversibly in the unchanged engine and rebuilt in 3 of 3 held-out folds. Its output equals the evaluator's answer. Frozen native CORA and matched extra search do not solve this task.
- **Why it is narrow.** The task was chosen in development by a failure autopsy, and it is an exposed public training task. A constructor that ignored the failure record found the same rule. So the run shows that CORA can **construct and verify** a missing rule. It does not show that diagnosing the failure **caused** the gain.
- **The offline demo** (`sh docs/summit_demo_20261023/run_demo.sh --real`) replays the recorded run and re-executes the constructed operation as standalone code. It is not a live rerun of the full engine.

7 Negative evidence and the measured bottleneck

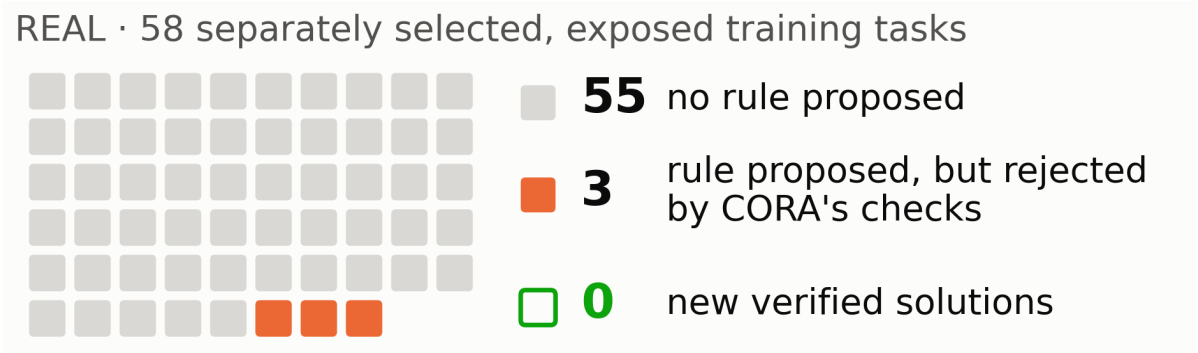


Figure 5: The 58-task generalization test of rule construction, one square per task, for the arm that used the failure record. No rule was proposed on 55 tasks; on 3, a rule was proposed but rejected (2 by the unchanged engine, 1 by the held-out rebuild); 0 verified.

- **The 58-task test.** 58 separately selected, still exposed training tasks, under a protocol frozen before the run. No construction arm (with the failure record, failure-blind, shuffled, contrast) produced a verified new answer: 0 of 58, with 0 wrong and 0 lost. The failure-blind arm: 52 no rule, 5 engine rejections, 1 held-out failure. Frozen native solved 1, which is not a construction result.
- **A restricted check on why.** An evaluator-only analysis, which reads the answers and is used only for diagnosis, asked whether a **single, uniformly applied** growth expression could explain the changed cells in any of native's seven views. For 50 of 58 tasks it could not. This supports an expressivity hypothesis inside that one class. It does not show that the tasks need new semantics or are impossible for CORA.
- **Fitting is not enough, measured.** On 60 exposed development tasks, CORA's object reasoner accepted 6 answers, all correct. On 26 tasks every rule that fit the demonstrations failed the test (figure 6).
- **Thin evidence, flagged.** CORA-v2 certified no answer on 118 exposed development tasks; its exploratory guesses were 4 correct and 1 wrong. On 00dbd492 its 1,365 consistent hypotheses agree and the guess is correct, but the colour for the largest frame is shown by one demonstration only. Hidden, that demonstration leaves the rebuilt rule without an answer, so v2 keeps the guess uncertified (figure 7).
- **Phase 2a (finished and evaluated, NO-GO).** On 40 freshly selected public training tasks, frozen native and matched search each certified 6 answers (5 correct, 1 wrong). CORA-v2 certified none; its 2 exploratory answers were correct, one of them new. The frozen GO rule needed 2 new correct answers, so the version was closed without retuning. On 33 of the 40 tasks no hypothesis in v2's language fit at all: the same bottleneck.

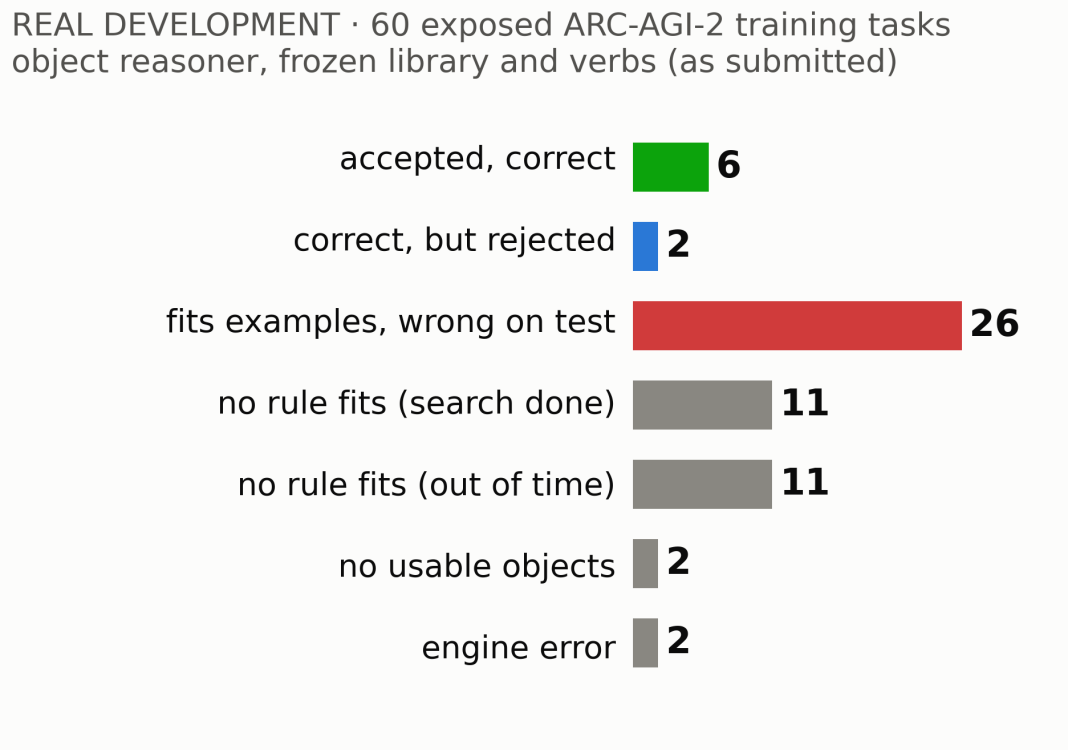


Figure 6: The 60-task audit of native CORA's object reasoner, as submitted, on exposed training tasks. This is one branch; the whole harness solved 12 of 60 over two attempts.

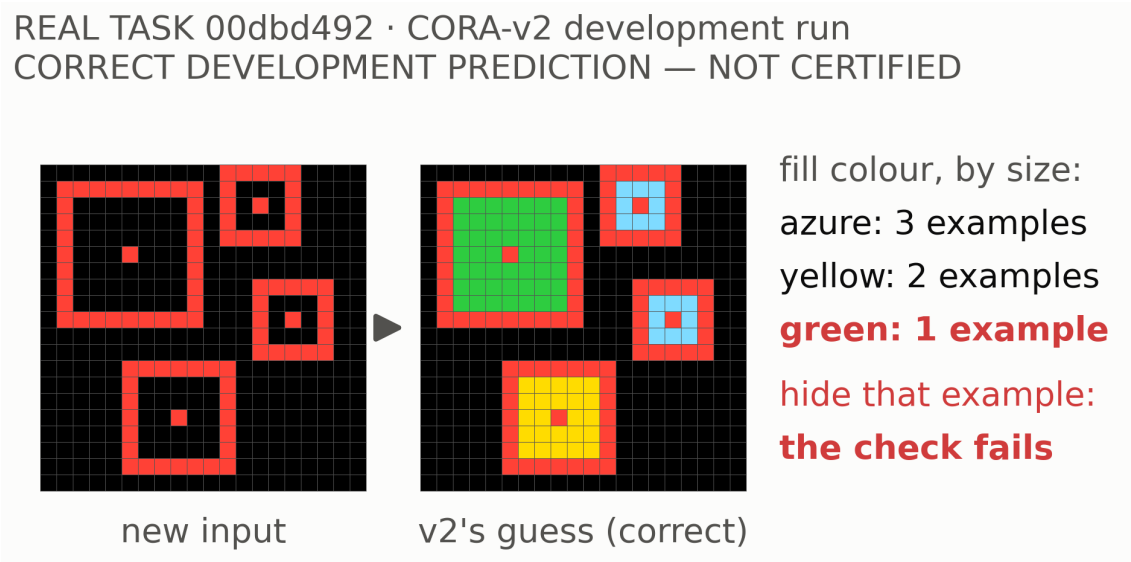


Figure 7: CORA-v2 on 00dbd492 (development run): a correct but uncertified prediction. The fill colour for the largest frame rests on one demonstration.

8 Why NVARC, and why neural learning

- **What was reproduced.** NVARC is the NVARC team's published ARC Prize 2025 method. I deployed their public notebook with six changes, all for deployment (compatibility, coverage, diagnostics, provenance, robustness). Unchanged: their Qwen3-4B checkpoint, their per-task test-time training, decoding, scoring and selection.
- **What Qwen learned in that method.** For each task, NVARC briefly trains a small adapter on the task's demonstrations (eight rotations and reflections × sixteen colour permutations), then decodes and scores candidate grids. This is learning within a task; the model emits grids, not programs.
- **Attribution.** 27.64% is the official public score of my deployment of their method (Kaggle 57160429). It is not a CORA result and not a Neural-CORA result. Native CORA and CORA with rule construction scored 0.00% (57144804, 57354496).
- **Why combining answers failed.** On 240 exposed stand-in tasks, NVARC got 248 of 259 outputs and CORA 47; CORA solved none that NVARC missed. Filling NVARC's empty second attempts with CORA's answers gained 0 and lost 1. On the hidden set CORA scored 0.00, so its grids could add nothing there.
- **An earlier neural negative.** A frozen, prompted Qwen2.5-1.5B that ranked CORA's seven segmentation views did not beat a fixed order (35 of 35 against 19), and failure traces made it worse (1 of 35, the same as shuffled traces).
- **Why a trained prior inside CORA is a different hypothesis.** It would be trained, not prompted. It would order a search space of thousands to millions of typed programs, where no fixed order can be best for everything, instead of choosing one of seven views. It uses no failure traces, and it must beat strong symbolic search under matched compute.

9 The proposed single Neural-CORA architecture

One inference algorithm, owned by CORA; not a router, an ensemble or a second-attempt filler (figure 8).

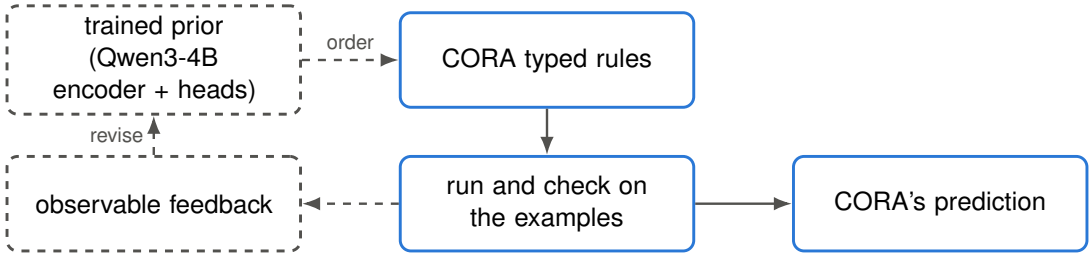


Figure 8: The proposed learning component inside CORA. Solid: CORA's existing typed rules, interpreter and prediction. Dashed: the trained prior and the feedback that would revise it; not yet validated.

- **CORA's typed state:** representation (a segmentation view), one or two first-match rules (selector and native action, additive or not), a default, and the evidence: which demonstrations fit, fold outcomes, rival predictions, cost.
- **The trainable prior:** the frozen NVARC Qwen3-4B checkpoint reads the demonstrations in its own grid format. A small trainable head turns its hidden states into scores over program components. The scores only **order** CORA's search.
- **Execution and selection:** the unchanged native interpreter runs each candidate; the same fixed rule picks two attempts for every arm. No neural grid reaches an attempt.
- **Fold-locality:** a prior adapted on all demonstrations cannot certify a fold; its proposals are labelled neural-assisted unless the adaptation is repeated without the held-out demonstration.
- **Status, 19 October:** the typed language compiles to native programs and runs through the native interpreter; CORA's own synthetic task generator (execution-validated, no third-party data) and the arms are implemented, with unit tests. A small engineering dry run is under way. **There is no**

result. The pilot protocol is frozen before any scored run.

10 How the idea could be falsified

Every arm uses the same inputs, the same interpreter and the same checker:

arm	what it is	what it controls
C0	native CORA's own inducer	the original system
C+	simplest-first search, twice the learned budget	extra symbolic search
C+H	C+ plus a hand-designed analysis of the changes	strong symbolic search
L0, L0s	the same prior untrained, or trained on shuffled targets	does training matter?
L1	the trained prior	the hypothesis
L1h	the same heads on hand-crafted features	does the neural encoder matter?
N0, N+	unchanged NVARC; NVARC with matched extra compute	the neural baseline (later stages)

Three outcomes must be kept apart: **candidate coverage** (a correct program is proposed at all), **exact accuracy** (the two attempts are right), and **diagnostic benefit** (authentic feedback chooses revisions better than blind or shuffled feedback). If the trained prior cannot beat C+ under matched resources on held-out task families, the result is NO-GO and the bottleneck is diagnosed before any larger model is trained.

11 People and the broader programme

- **Toni Kazic.** The general question at the top of the poster was discussed with Toni in a recorded conversation (transcript: `voice_note/arc_conversatn.md`). The poster's wording is the author's; Toni's preferred wording is to be confirmed.
- **Chris.** The 2025 ARC work with Chris (the slides list Christian Fluharty, Computer Science, as co-presenter; identity and naming to be confirmed) used LLM proposals with a program language, which this work builds on in spirit.
- **Pierce Taylor.** A joint project with Pierce and Toni exists (`sb_class_project`). Its description, and any biological test of a revision procedure, await Pierce's review. No conclusion from it is reported here.
- **What could transfer** is the revision policy and the verification discipline, not ARC operations. Biology remains future, untested work.

12 Reproduction, terms and responsible claims

- **Figures:** `python scripts/summit_visual_figures.py` (stops if any pinned input's hash differs); provenance in `figures/PROVENANCE.json`.
- **This explainer and the poster:** `sh scripts/build_summit_org.sh all` exports each `.org` to LaTeX with Emacs and compiles that export; `sh scripts/build_summit_org.sh selftest` proves the `.org` files are the real sources. Details: `ORG_TO_PDF_README.md`.
- **Records:** the claim ledger; `records/CORA_REASONING_V2_PHASE2A_EXECUTION_20261019.md`; `records/NEURAL_CORA_ARCHITECTURE_AUDIT_20261019.md`.

Terms kept off the poster. *Leave-one-out* (LOO): rebuild a rule with one demonstration hidden and check that it predicts it. *Certified*: unique and rebuilt in every fold. *Exploratory*: fits every demonstration but is not certified. *FHR*: failure-conditioned hypothesis revision. *Grammar* or *rule language*: what a

system can express at all. *K2* and *Step B*: a separate CORA experiment on inventing new operations, not used here. *Oracle*: an evaluator-only analysis that reads answers for diagnosis only. *OOD*: tasks unlike those used in development. *Causal claim*: a component caused a gain, shown only by a controlled comparison. *Exposed task*: already examined in development, so results on it are in-sample. *Version space*: every hypothesis consistent with the demonstrations.

Credits and disclosure. Authorship and credits are pending review by Toni, Chris and Pierce. AI coding assistance was used for parts of the implementation and documentation; the research, decisions and claims are the author's.